

GOLDTRADINGBOT

Technical Audit & Development Report

Instruments: XAUUSD • BTCUSD
Framework: Python + MetaTrader 5 + MQL5

Report Date	April 08, 2026
Version	1.0
Status	Audit Complete — Fixes Implemented
Language	Python 3.x / MQL5
Broker API	MetaTrader 5 (MT5)

This document is a comprehensive technical audit and development log of the GoldTradingBot project, covering strategy logic, EA bridge architecture, backtesting integrity, bug fixes, and optimization methodology.

Table of Contents

1. Executive Summary

2. Project Overview

- 2.1 System Architecture
- 2.2 Technology Stack
- 2.3 File Structure

3. Trading Strategy Deep-Dive

- 3.1 Overview & Philosophy
- 3.2 Multi-Timeframe Framework (H4 + M15)
- 3.3 Signal Generation Step-by-Step
- 3.4 Limit Order Logic: End-to-End
- 3.5 Order Cancellation Scenarios

4. Code Audit: EA Bridge & Expert Advisor

- 4.1 Audit Scope
- 4.2 Critical Bug: Encoding & File Locking
- 4.3 Critical Bug: Infinite CANCEL_ALL Spam
- 4.4 Critical Bug: Duplicate Limit Orders
- 4.5 Medium Bug: Ignored Order Expiration
- 4.6 Bias Assessment

5. Look-Ahead Bias Audit

- 5.1 What is Look-Ahead Bias?
- 5.2 Audit Results: Three Safe Patterns
- 5.3 Verdict

6. The `bos_time` Forward-Fill Bug

- 6.1 Root Cause Analysis
- 6.2 Why the Bridge Went Silent
- 6.3 The Fix

7. Bridge Architecture: Event-Based vs State-Based

- 7.1 The Original Event-Based Architecture
- 7.2 The Silent Command Failure
- 7.3 The Upgraded State-Based Architecture

8. Runtime Diagnostics & Fixes

- 8.1 Windows QuickEdit Mode vs File Locking
- 8.2 CSV Engine Optimization

9. Walk-Forward Optimization

- 9.1 Why Grid Search Overfits
- 9.2 Walk-Forward Optimization Explained
- 9.3 Implementation

10. Parameter Sensitivity & Heatmap

11. Visual Replay Engine

12. Backtest vs Live Discrepancy Analysis

13. Recommendations & Future Work

14. Glossary

1. Executive Summary

GoldTradingBot is a fully automated algorithmic trading system designed to trade XAUUSD (Gold) and BTCUSD using a Smart Money Concepts (SMC) strategy. It consists of a Python-based analysis engine that communicates with a MetaTrader 5 Expert Advisor (EA) via CSV files, allowing the two components to work together across the Python and MT5 environments.

This report documents the complete development history, including a thorough code audit, all bugs found and fixed, a look-ahead bias review, optimization methodology, and architectural improvements made to ensure the bot operates reliably in live trading conditions.

Key Findings at a Glance

Area	Finding	Severity	Status
EA Bridge (ea_bridge.py)	UTF-8 vs UTF-16 encoding mismatch + file locking	Critical	Fixed
EA Bridge	Infinite CANCEL_ALL spam (1,440/day)	High	Fixed
EA Bridge	No check against open positions — duplicate limit orders	High	Fixed
myEA.mq5	max_pending_bars ignored — ORDER_TIME_GTC hardcoded	Medium	Fixed
strategy.py	bos_time not forward-filled — setup vanishes after 15 min	High	Fixed
ea_bridge.py	Event-based architecture — silent command failures possible	High	Upgraded
ea_bridge.py	pd.read_csv engine='python' — caused 15-min file-lock hangs	Medium	Fixed
backtest.py	Look-ahead bias check	Info	None Found
optimizer.py	Grid search overfitting risk	Medium	Upgraded to WFO

Overall Verdict: The strategy logic is mathematically sound with zero look-ahead bias. All critical bugs have been resolved. The system is now significantly more robust for live trading.

2. Project Overview

2.1 System Architecture

GoldTradingBot is built on a two-process architecture. Python acts as the 'Brain' — it performs all mathematical analysis, signal detection, and decision-making. MetaTrader 5 acts as the 'Muscle' — it handles live market data, broker communication, and order execution. These two processes communicate through shared CSV files on the local disk.

PYTHON ENGINE (Brain)	CSV Files	METATRADER 5 (Muscle)
strategy.py Signal detection (BOS, sweeps, Fib)	<-- ag_rates_XAUUSD.csv ag_rates_BTCUSD.csv ag_desired_state.csv -->	myEA.mq5 Exports candle data Places/cancels orders
ea_bridge.py Polls every 60s Writes desired state		Broker / Live Market

Figure 1: High-level system architecture

2.2 Technology Stack

Component	Technology	Purpose
Signal Engine	Python 3.x	Market analysis, signal generation, bridge logic
Data Library	pandas, numpy	Time-series manipulation, vectorized calculations
Broker API	MetaTrader5 (Python package)	Fetch live/historical OHLCV data from MT5
Expert Advisor	MQL5	Executes orders in MT5 based on Python commands
Communication	CSV files (shared disk)	Rates export, desired state, command handoff
Backtesting	Custom backtest.py	Simulate trades on historical data
Optimization	Walk-Forward (optimizer.py)	Parameter tuning without overfitting
Visualization	HTML + JavaScript	Animated visual replay of backtests
Heatmap	seaborn (heatmap.py)	Parameter sensitivity analysis

2.3 File Structure

- main.py** — Entry point. CLI interface for selecting symbols, date range, and replay options.
- strategy.py** — Core signal engine. Detects BOS, liquidity sweeps, and calculates Fibonacci entry/SL/TP.
- backtest.py** — Historical simulation engine with trade logging, plotting, and Monte Carlo simulation.
- data_loader.py** — MT5 connection, OHLCV data fetching, and symbol specification retrieval.
- ea_bridge.py** — Polling loop. Reads rates CSV, runs strategy, writes desired state to MT5.
- ea/myEA.mq5** — MQL5 Expert Advisor. Exports candle data and reconciles desired state with live orders.
- optimizer.py** — Walk-Forward Optimization engine for robust parameter selection.

heatmap.py — Parameter sensitivity heatmap across HTF/ETF window combinations.

visual_replay.py — Generates interactive HTML animated replay with PnL, return %, and days counter.

3. Trading Strategy Deep-Dive

3.1 Overview & Philosophy

The bot uses a Smart Money Concepts (SMC) strategy. Rather than relying on traditional indicators like RSI or MACD, SMC focuses on how institutional traders ('smart money') move the market. The core idea is to trade in the direction of the dominant trend (identified on a higher timeframe) and enter precisely when the market has swept retail stop-losses and confirmed a reversal on a lower timeframe.

3.2 Multi-Timeframe Framework (H4 + M15)

The strategy uses two timeframes working together. The H4 (4-hour) chart is the 'big picture' view. It tells the bot whether the overall market trend is bullish or bearish. The M15 (15-minute) chart is the 'fine detail' view. Once the H4 gives a direction, the bot watches the M15 for precise entry opportunities.

Timeframe	Role	Key Parameters
H4 (4-Hour)	Anchor / Bias Filter	anchor_swing_window = 7 bars HTF warmup = 60 days Determines bullish or bearish bias
M15 (15-Minute)	Execution / Entry	ETF warmup = 360 candles (~3.75 days) max_pending_bars = 96 bars (24 hours) Entry, SL, TP calculated here

3.3 Signal Generation Step-by-Step

When all conditions align, the strategy creates a trading setup with four exact price levels: Entry (where the limit order sits), Stop Loss (maximum acceptable loss), Take Profit (target), and BOS Time (timestamp of the structural confirmation).

Step 1 — H4 Bias

The bot scans the H4 chart to identify the most recent swing high and swing low. If the latest confirmed BOS on H4 is bullish (price broke above a recent swing high), the bias is set to Long. If bearish (broke below a swing low), bias is set to Short. The bot will only take trades in this direction.

Step 2 — M15 Liquidity Sweep

On the M15 chart, the bot watches for price to move beyond a previous swing point — either wicking above a recent high (sweeping buy-side liquidity) or below a recent low (sweeping sell-side liquidity). This is the market's way of triggering retail stop-losses before reversing.

Step 3 — Break of Structure (BOS)

After the sweep, the bot waits for price to reverse and close decisively beyond an internal structure level in the direction of the H4 bias. This closing candle is the BOS confirmation candle — the formal signal.

Step 4 — Fibonacci Entry Calculation

Once BOS is confirmed, the bot calculates entry, SL, and TP levels. The entry is placed at the 0.618 Fibonacci retracement of the BOS leg (i.e., price should retrace back 61.8% of the move before continuing). The Stop Loss is placed at the extreme of the liquidity sweep. The Take Profit is placed at the origin of the BOS move.

Step 5 — Signal Persistence (ffill)

The setup is 'forward-filled' across the next `max_pending_bars` (default 96) M15 candles. This keeps the setup visible to the bridge for up to 24 hours while the market retraces to the entry level.

3.4 Limit Order Logic: End-to-End

The flow from signal detection to an active limit order on the broker's servers involves five distinct stages across two processes:

Stage	Process	What Happens
1. Signal Detection	strategy.py	SMC conditions met → entry/SL/TP/bos_time calculated
2. Signal Persistence	strategy.py	ffill propagates setup across 96 M15 candles
3. Bridge Polling	ea_bridge.py	Every 60s: reads rates CSV, runs strategy, reads <code>iloc[-2]</code>
4. State Declaration	ea_bridge.py	Writes desired state to <code>ag_desired_state.csv</code>
5. EA Reconciliation	myEA.mq5	Reads desired state, checks reality, places/cancels order

3.5 Order Cancellation Scenarios

A pending limit order can be cancelled by four different scenarios. Importantly, the upgraded State-Based architecture means the EA continuously reconciles these conditions — there is no single point of failure:

- 1. Strategy Time-out:** After 96 M15 candles (24 hours), ffill exhausts. `strategy.py` returns NaN. The bridge updates `ag_desired_state.csv` to 'no order'. The EA cancels the pending order.
- 2. Internal Dead-Man Switch (MT5):** When the EA places an order, it sets an automatic expiration timestamp inside MT5. Even if Python crashes or the computer turns off, MT5's internal clock will cancel the order when it expires. Duration: `max_pending_bars × bar_duration` seconds.
- 3. Setup Replacement:** If a new and different BOS setup forms while the old one is still pending, the EA cancels all existing orders before placing the new limit order.
- 4. Bias Flip:** If the H4 trend reverses (from bullish to bearish or vice versa), `strategy.py` invalidates the current setup and returns NaN. The bridge updates the desired state to empty, and the EA cancels the orphaned order.

4. Code Audit: EA Bridge & Expert Advisor

4.1 Audit Scope

The audit covered `ea_bridge.py` (Python bridge script) and `myEA.mq5` (MQL5 Expert Advisor). The `live_trader.py` file was excluded from the audit as it is no longer used for trade execution. The audit assessed execution logic, file I/O safety, bias prevention, and order management correctness.

The overall architecture — Python producing a signal and the EA consuming it — is conceptually sound. The issues found were in the implementation details of file communication and order management.

4.2 Critical Bug: Encoding Mismatch & File Locking

Severity: CRITICAL | Status: Fixed

The Problem

MetaTrader 5 uses UTF-16 encoding with a tab (`\t`) delimiter by default when writing CSV files. Python's pandas library writes UTF-8 with a comma (`,`) delimiter. These two defaults are completely incompatible — when Python attempted to read the rates file that MT5 wrote, it would misread the characters and fail to parse the data correctly. Additionally, the EA's `FileOpen` call did not include file-sharing flags (`FILE_SHARE_READ | FILE_SHARE_WRITE`). This meant MetaTrader permanently locked the CSV file while the EA was active. When Python attempted to open the same file 60 seconds later, the operating system would block it, causing Python to hang indefinitely.

The Fix

The EA's `FileOpen` call was updated to include file-sharing flags. The Python bridge was updated to explicitly specify `encoding='utf-16'` and `sep='\t'` when reading MT5-exported files, and `encoding='utf-8'` with `sep=','` when writing command files that Python controls.

4.3 Critical Bug: Infinite CANCEL_ALL Spam

Severity: HIGH | Status: Fixed

The Problem

In the original `ea_bridge.py`, whenever `strategy.py` returned no active setup (setup is `None`), the bridge would blindly append a `CANCEL_ALL` command to the `ag_commands.csv` file. Since the bridge polls every 60 seconds, this created 1,440 `CANCEL_ALL` commands per day under normal idle conditions. This caused the CSV file to bloat endlessly and forced the EA to waste processing cycles continuously reading an extremely large file.

The Fix

The bridge was refactored to use state-based logic. Instead of appending commands, it overwrites a single `ag_desired_state.csv` file with the current desired state. When there is no setup, it writes a single `NONE` state — not a growing list of cancellations.

4.4 Critical Bug: Duplicate Limit Orders

Severity: HIGH | Status: Fixed

The Problem

The EA checked `OrdersTotal()` to prevent placing a duplicate pending order. This check is correct while the limit order is sitting on the books as a pending order. However, the moment that limit order is hit by price and becomes an active open position, it disappears from `OrdersTotal()` entirely. If Python re-sent the `PLACE_LIMIT` command at that point, the EA would see zero pending orders and blindly place a second limit order on top of the existing active trade — effectively doubling the position size and risk.

The Fix

The EA's safety check was expanded to also query `PositionsTotal()` for the current symbol and magic number. Now, before placing any order, the EA verifies there is neither an existing pending order nor an open position for that instrument.

4.5 Medium Bug: Ignored Order Expiration Variable

Severity: MEDIUM | Status: Fixed

The Problem

Python dynamically supplied a `max_pending_bars` variable through the command CSV to tell the EA how long to keep a limit order alive. However, `myEA.mq5` received this value into a variable (`h_maxpb`) but then completely discarded it. The order duration was hardcoded to `ORDER_TIME_GTC` (Good Till Cancelled). This meant orders could theoretically sit on the broker's server for weeks, risking accidental execution if the Python script crashed.

The Fix

The EA was updated to convert `max_pending_bars` to an actual expiration timestamp using MT5's `TimeCurrent()` + (`max_pending_bars * bar_duration_in_seconds`), and pass this as `ORDER_TIME_SPECIFIED` to the broker. This creates a true server-side dead-man's switch.

4.6 Bias Assessment

Bias Type	Assessment	Details
Look-Ahead Bias	NONE FOUND	<code>df.iloc[-2]</code> guarantees trading on the most recently COMPLETED candle only. Future
Signal Persistence Bias	RESOLVED	Previously, <code>bos_time</code> was not forward-filled, causing setups to appear valid for only c
Execution Bias	NONE FOUND	Signal generation and order execution are cleanly separated. No signal is acted upon

5. Look-Ahead Bias Audit

5.1 What is Look-Ahead Bias?

Look-ahead bias occurs when a backtesting system uses data from the future to make trading decisions in the past. For example, if the bot 'knows' at 9:00 AM that the market will close higher at 4:00 PM and uses this knowledge to buy at 9:00 AM, the backtest results are completely unrealistic — that information would never be available in real-time trading. Look-ahead bias is one of the most common and dangerous flaws in algorithmic trading systems because it produces spectacular backtest results that completely fall apart in live trading.

5.2 Audit Results: Three Safe Patterns

Pattern 1: Anchor Confluence Delay (.shift(1))

When merging the H4 bias onto the M15 execution timeframe, the code correctly uses `.shift(1)` on the anchor context before joining with `pd.merge_asof(direction='backward')`. This is critical: it guarantees that the M15 chart can only 'see' an H4 candle that has fully and completely closed. The currently-forming H4 candle is never accessible to the signal generator.

Pattern 2: Swing Confirmation (find_swings)

The `find_swings` function uses a lookback window to identify swing highs and lows. Crucially, it uses `.shift(window)` to delay the `is_swing_high` and `is_swing_low` flags. This means a bar can only be officially confirmed as a swing point after the required number of subsequent confirming candles have fully closed. It is mathematically impossible for the bot to trade based on a swing that has not yet been confirmed by real historical price action.

Pattern 3: Trigger Safety (pending_active_from_i)

In the backtest engine's `run_backtest` loop, after a signal is detected on candle `i`, the code sets `pending_active_from_i = i + 1`. This ensures that the limit order only becomes 'active' (eligible to be hit) on the NEXT candle. The bot can never fill a trade on the same bar that generated the signal — which would require knowing how high or low that candle would close.

5.3 Verdict

VERDICT: Zero Look-Ahead Bias Detected. The backtesting architecture is rigorously causal. All three core safety patterns are correctly implemented. The `.shift(1)` anchor delay, historical swing confirmation, and `i+1` trigger safety work together to guarantee no future data is ever used in signal generation or trade execution.

6. The bos_time Forward-Fill Bug

Severity: HIGH | Files: strategy.py, ea_bridge.py | Status: Fixed

6.1 Root Cause Analysis

This bug was the root cause of the bridge silently failing to place live orders even when a valid backtest signal existed. The bug was subtle: all of the primary signal columns were correctly propagated, but one required field was missed.

In strategy.py, there is a list of columns that must be 'forward-filled' to keep a setup visible across multiple candles:

```
cols_to_ffill = ['setup_id', 'setup_dir', 'entry_level', 'sl_level', 'tp_level']
```

And in ea_bridge.py, there was a strict validation check:

```
bos_time = row.get('bos_time', pd.NaT) if pd.isna(bos_time): return None # SETUP INVISIBLE
```

The bridge required bos_time to be non-null to consider a setup valid. However, bos_time was never added to the forward-fill list. This meant it only appeared on the exact candle where the BOS formed, and was NaN (empty) on every subsequent candle.

6.2 Why the Bridge Went Silent

The following table illustrates exactly what the data looked like before the fix. The bridge reads df.iloc[-2] (the last completed bar). It had only a single 15-minute window to detect the setup:

Time	setup_id	entry_level	bos_time	Bridge Sees Setup?
05:30 AM (BOS candle)	20	71477	05:30 AM	YES (only this window)
05:45 AM	20	71477	NaN (empty)	NO — returns None
06:00 AM	20	71477	NaN (empty)	NO — returns None
06:15 AM	20	71477	NaN (empty)	NO — returns None

Table: Data state before the fix. The bridge returned None on every bar after 05:30 AM.

6.3 The Fix

The fix was a single line addition in strategy.py:

```
# Before (Missing bos_time): cols_to_ffill = ['setup_id', 'setup_dir', 'entry_level', 'sl_level', 'tp_level'] # After (bos_time added): cols_to_ffill = ['setup_id', 'setup_dir', 'entry_level', 'sl_level', 'tp_level', 'bos_time']
```

With this change, bos_time is now propagated across all 96 candles of the setup's pending window, making the setup continuously visible to the bridge for the full 24-hour validity period.

Impact: This single missing column was responsible for the bot appearing to run correctly while silently failing to place any live orders after the initial 15-minute detection window. It was the primary cause of the backtest vs live trade discrepancy.

7. Bridge Architecture: Event-Based vs State-Based

7.1 The Original Event-Based Architecture

The original bridge used an 'Event-Based' or 'Command Log' approach. Python would append a `PLACE_LIMIT` command to `ag_commands.csv` whenever a new setup was detected. The EA would read this growing log and execute commands as it found them.

Analogy: Think of this as a To-Do List. Python writes a note: 'Place BTC order.' The EA reads the note and crosses it off. If the notebook is lost, the EA misses the page, or the broker returns an error — the order is gone forever.

7.2 The Silent Command Failure

A concrete failure was observed on April 8, 2026 at 06:15 AM. A BTCUSD setup was detected at 05:30 AM. The bridge sent the `PLACE_LIMIT` command at approximately 05:46 AM and updated its internal memory to 'command sent.' However, the EA did not successfully place the order — possibly due to a brief file-access conflict or broker stop-level restriction at that exact moment. Because the bridge believed its job was done, it never retried. The order was silently lost. The backtest showed a winning BTCUSD trade at 06:15 AM. The live EA showed nothing.

7.3 The Upgraded State-Based Architecture

The system was upgraded to a 'State-Based' or 'Dashboard' approach. Instead of a growing command log, the bridge maintains a single file (`ag_desired_state.csv`) that always reflects the current desired state of the world.

Analogy: Think of this as a Dashboard. Python's dashboard always displays: 'For BTCUSD: I want a Short Limit at 71477.' The EA checks this dashboard every minute. If reality doesn't match — it fixes it. If the broker was busy and the order didn't place, the EA will see it is still missing next minute and try again automatically.

Property	Event-Based (Old)	State-Based (New)
File type	Append-only command log	Single overwritten state file
File growth	Grows indefinitely (1,440 entries/day)	Always exactly one row per symbol
Missed command	Lost forever — no retry	EA retries every 60 seconds
Internet cut-out	Order stays on books (risk)	EA reconciles on reconnect
Manual deletion	Order stays deleted	EA re-replaces the order within 60s
Broker error	Silently fails	EA retries until success
Self-healing	No	Yes

8. Runtime Diagnostics & Fixes

8.1 Windows QuickEdit Mode vs File Locking

A symptom was observed where the Python console stopped printing output for 15 minutes (from 10:54 AM to 11:09 AM) and then resumed with only the latest log, skipping all intermediate cycle logs. Initial analysis suggested Windows QuickEdit Mode (a feature that freezes the console when the user accidentally clicks inside the terminal window). However, the pattern of skipped logs — rather than a flood of buffered logs on resume — confirmed the actual cause was file I/O blocking.

Root cause: pandas read_csv with engine='python' and sep=None uses a file format guessing algorithm. When MT5 or an older copy of the EA held a hard lock on the CSV file, the Python engine entered an OS-level wait state with no timeout. It did not crash, raise an error, or move on — it simply waited. The 15-minute pause was a single loop iteration stuck on one read_csv call.

Key Distinction:

Symptom Pattern	Cause
Console resumes with all buffered logs (flood of entries)	QuickEdit Mode freeze — program was paused
Console resumes with only the latest entry (entries skipped)	File I/O blocking — program was stuck on one line

8.2 CSV Engine Optimization

The following changes were made to ea_bridge.py to prevent file I/O blocking:

- Fast Engine:** Changed from engine='python' to engine='c'. The C engine is 5-10x faster and does not use guess-based parsing.
- Explicit Separator:** Changed from sep=None to sep=','. Eliminates the format-sniffing step entirely.
- PermissionError Catch:** Added an explicit except (PermissionError, IOError) block. If the file is locked, Python now logs a warning, waits 0.5 seconds, and retries — rather than hanging silently.
- Cycle Heartbeat Logs:** Added 'Cycle complete. Sleeping...' log at the end of each loop. If 'Cycle start' appears but 'Cycle complete' never follows, the exact symbol causing the hang can be identified.

9. Walk-Forward Optimization

9.1 Why Grid Search Overfits

The original optimizer.py used a Grid Search approach: it tested every possible combination of parameters (Fibonacci levels, HTF swing window, ETF swing window) across the entire historical dataset and returned the combination with the best backtest performance.

The fundamental problem with Grid Search on a single dataset is overfitting. The algorithm will find parameters that perfectly avoided stop-losses by one pip in the historical data — not because the parameters are robust, but because they happened to fit the specific quirks and noise of that particular price history. In live trading, these parameters almost invariably fail because the future does not replay the past exactly.

9.2 Walk-Forward Optimization Explained

Walk-Forward Optimization (WFO) solves the overfitting problem by never allowing the optimizer to see the test data during training. It simulates the real-world process of periodically re-optimizing a live strategy.

- Train Window (In-Sample, 6 months):** The optimizer runs a grid search on the first 6 months of data to find the best-performing parameter set for that period.
- Test Window (Out-of-Sample, 3 months):** The best parameters are immediately 'locked in' and tested blindly on the next 3 months of data — data the optimizer has never seen. No further tuning is allowed.
- Walk Forward:** The entire window slides forward by 3 months and the process repeats: train on months 4-9, test on months 10-12, and so on.
- Final Assessment:** All out-of-sample test periods are stitched together to form a final Out-of-Sample Portfolio result. This represents the true expected performance of the strategy in live conditions.

The Golden Rule: If the final Out-of-Sample portfolio shows positive PnL and a reasonable win rate, the strategy is considered robust and non-overfit. A strategy that performs well in-sample but poorly out-of-sample is overfit and should not be traded live.

9.3 Implementation

The optimizer.py file was completely rewritten to implement WFO with the following default configuration:

Parameter	Default Value	Description
Train window	6 months	In-sample optimization period per fold
Test window	3 months	Out-of-sample blind test period per fold
Step size	3 months	How far the window slides each fold
Parameter grid	Fib levels, HTF window, ETF window	Variables being optimized
Selection metric	Net PnL	Criterion for choosing best parameters in train

10. Parameter Sensitivity & Heatmap Analysis

The heatmap.py tool provides a visual sanity check on parameter robustness. It runs the backtest engine in rapid succession across a wide grid of HTF window and ETF window combinations, then plots the results as a 2D color-coded matrix using the seaborn library.

How to Interpret the Heatmap

Green Cluster (Robust): A group of adjacent green cells indicates that similar parameter values all produce profitable results. The strategy is not sensitive to small changes — it works across a range of settings. This is a sign of robustness.

Single Green Cell (Overfit): If only one or two cells are green, surrounded by red, the strategy only works for one very specific combination. This is a strong signal of overfitting to historical noise. The strategy should not be traded live.

All Red (Strategy Failure): If the entire heatmap is red, the strategy may not have a viable edge in the selected time period or instrument. Reconsider the underlying logic.

Workflow

The recommended workflow is to use WFO (Section 9) to find candidate parameter sets, then use the heatmap to visually confirm the winning parameters sit inside a green cluster. If they do, confidence in the strategy's robustness is high. Run the heatmap with: `python heatmap.py`

11. Visual Replay Engine

The `visual_replay.py` module generates an interactive HTML file that animates the entire backtest on a candlestick chart. It is integrated directly into the main backtest pipeline and can be enabled at runtime.

Features

Feature	Description
Animated Candlestick Chart	Price forms bar by bar at an adjustable speed
Play / Pause / Step Controls	Full playback control including single-bar stepping
Speed Slider	Adjust from slow-motion analysis to rapid review
Limit Order Visualization	Dashed lines appear when a limit order is placed (Entry, SL, TP)
Trade Entry/Exit Markers	Lines turn solid when the limit is hit; color-coded green (win) or red (loss)
Live PnL Counter	Displays current account equity including floating unrealized P&L
Return % Display	Percentage return shown in green when profitable, red when in drawdown
Days Past Counter	Tracks how many calendar days have elapsed in the backtest period

How to Use

Run `python main.py` and when prompted, answer 'y' to 'Generate Animated Visual Replay.' The replay HTML file is saved to the `backtest_results` folder and can be opened in any modern web browser. No internet connection is required as all chart rendering is done in self-contained JavaScript.

12. Backtest vs Live Discrepancy Analysis

On April 7, 2026, a significant discrepancy was observed between backtest results and live EA execution. The backtest showed one winning XAUUSD trade and one losing BTCUSD trade. The live EA showed multiple losing XAUUSD trades and mixed BTCUSD results. Root cause analysis identified three distinct problems:

Problem 1: The Broken Record (Repeated Losses)

The original bot had no persistent memory between loop iterations. When a trade hit the Stop Loss and closed, the position disappeared from MT5. On the next cycle (60 seconds later), the bridge re-detected the same technical setup, saw no open position, and placed the same limit order again. This cycle repeated until the market moved far enough to invalidate the setup. The backtest only ever enters each unique setup once, creating a fundamental live/backtest mismatch. Fix: A persistent tracking file (`last_bos_SYMBOL.txt`) was implemented to record the `bos_time` of every setup that was acted upon, preventing re-entry on the same signal.

Problem 2: The Late Entry (Entering Dead Trades)

On XAUUSD, the winning move happened very quickly. If the bridge was restarted or the script was briefly stopped during the exact window when price was at the entry level, the bot would wake up, detect the technically-still-valid setup, and place the order — but price had already moved through the entry level and was heading back toward the SL. Fix: The bridge was updated to validate that the entry level has not already been touched before placing new orders.

Problem 3: The Time Window Mismatch

The backtest was configured to start at 00:00 on April 7. A winning BTCUSD trade the EA took actually occurred on April 6 in the late evening. The backtest simply didn't exist at that time and could not record it. This is not a bug — it is a configuration difference. Running the backtest with the exact same time range as the EA's operation period resolves this discrepancy.

13. Recommendations & Future Work

Immediate — Recompile EA After Changes

Any time myEA.mq5 is modified, it must be recompiled in MetaEditor (press F7). Running an outdated .ex5 binary with new source code will cause the EA to behave based on the old logic, making debugging extremely difficult.

Immediate — Disable Windows QuickEdit Mode

Although not the primary cause of the 15-minute hang, QuickEdit Mode can still accidentally pause the bot. Disable it via right-click on the console title bar > Properties > Options tab > uncheck QuickEdit Mode.

Short-Term — EA State Confirmation Logging

Add explicit MT5 order placement confirmation logs to the EA, writing the result (success or failure with error code) back to a status file. The bridge can then read this file to confirm the order was actually placed, rather than assuming success.

Short-Term — Multi-Symbol Heatmap

Run heatmap.py against both XAUUSD and BTCUSD simultaneously and compare cluster positions. Parameters that form a green cluster on both instruments are significantly more robust than those that only work on one.

Medium-Term — Slippage & Commission Calibration

Calibrate the backtest's spread_points, slippage_points, and commission_per_unit against actual filled trade data from the broker. Overly optimistic cost assumptions inflate backtest PnL and create unrealistic performance expectations.

Medium-Term — Monte Carlo Confirmation

After each WFO run, apply the Monte Carlo simulation (built into backtest.py) to the out-of-sample trades. If 80%+ of 1,000 random re-orderings of those trades end profitably, the strategy is statistically robust.

Long-Term — Live Trade Database

Replace the CSV-based tracking files (last_bos_SYMBOL.txt) with a lightweight SQLite database. This provides better atomicity, easier querying, and reduced risk of file corruption under concurrent access.

Long-Term — Alert System

Add a notification mechanism (email, Telegram, or desktop notification) when the bridge places an order, when a trade closes, or when the bridge encounters an error. This allows monitoring the bot without keeping the console open.

14. Glossary

BOS (Break of Structure)

A price candle that closes beyond a recent swing high or low, confirming a change in short-term market direction.

SMC (Smart Money Concepts)

A trading methodology based on identifying how institutional traders accumulate and distribute positions, characterized by liquidity sweeps and structural breaks.

Liquidity Sweep

A rapid price move beyond a swing point that triggers retail stop-loss orders before reversing. Also called a 'stop hunt.'

Fibonacci Retracement

A mathematical tool based on the Fibonacci sequence. The 0.618 (61.8%) level is widely used as an entry point after a directional move.

ffill (Forward Fill)

A pandas operation that propagates the last known non-null value of a column forward in time, keeping data visible across subsequent rows.

Look-Ahead Bias

Using future data to make past trading decisions, producing unrealistically good backtest results that fail in live trading.

WFO (Walk-Forward Optimization)

An optimization methodology that trains parameters on one period and tests them on a subsequent unseen period, repeated in rolling windows.

MQL5

The programming language used by MetaTrader 5 for Expert Advisors (automated trading scripts).

Expert Advisor (EA)

An automated trading program that runs inside MetaTrader 5.

ORDER_TIME_GTC

Good Till Cancelled — an MT5 order type that keeps a pending order active indefinitely until manually cancelled.

ORDER_TIME_SPECIFIED

An MT5 order type with a specific expiration timestamp, after which the broker automatically cancels the order.

State-Based Architecture

A system design where components communicate by declaring their desired state rather than sending one-time commands. Enables self-healing and automatic retries.

iloc[-2]

In pandas, this accesses the second-to-last row of a dataframe — the most recently COMPLETED candle (as the last candle, `iloc[-1]`, is still forming).

Monte Carlo Simulation

A statistical technique that runs thousands of random re-orderings of historical trades to assess the probability distribution of possible outcomes.

H4 / M15

Abbreviations for 4-hour and 15-minute candlestick chart timeframes respectively.

NaN

Not a Number. In pandas, used to represent missing or undefined data values.

End of Report — GoldTradingBot Technical Audit & Development Report v1.0

Generated on April 08, 2026